

Evaluating Bit-Width Trade-offs for a Fixed-Point Radix-2 DIT FFT Implementation on FPGA

Hoang Ngoc Gia Bao
FPT University

Ho Chi Minh City, Vietnam
baohoang037@gmail.com

Tran Ho Khanh Tan
FPT University

Ho Chi Minh City, Vietnam
tantran030306@gmail.com

Nguyen Trong Hung
FPT University

Ho Chi Minh City, Vietnam
nguyentronghung2124@gmail.com

Nguyen Que Vu
FPT University

Ho Chi Minh City, Vietnam
lolgamervu@gmail.com

Nguyen Viet Anh
FPT University

Ho Chi Minh City, Vietnam
nguyenvietanh29060@gmail.com

Abstract—Fixed-point arithmetic is widely used in FPGA-based signal processing because it consumes fewer hardware resources than floating-point. The drawback is that shortening the data word length introduces quantization noise that builds up stage by stage and degrades output quality. This paper reports a complete, verified design flow for studying this trade-off in a 1024-point fixed-point Radix-2 Decimation-in-Time (DIT) Fast Fourier Transform (FFT) on the Xilinx Artix-7 FPGA. A bit-accurate Python model of the FFT is built to capture all finite-word-length effects, including twiddle factor quantization, post-multiply rounding, and per-stage overflow scaling. A parameterized Verilog RTL design is co-simulated against this model and shown to be bit-exact across all 1024 output bins for five signal types and two word lengths. Synthesis on the Artix-7 (xc7k70tffbv676-1) shows that at $W = 12$ bits the design reaches 34.4 dB Signal-to-Quantization-Noise Ratio (SQNR) for a single-tone input while using only five DSP48E1 blocks and two Block RAM tiles. Among all configurations tested, the 12-bit setting offers the best balance between accuracy and hardware cost.

Index Terms—FFT, fixed-point arithmetic, FPGA, SQNR, bit-width trade-off, Radix-2 DIT, finite word-length, Artix-7

I. INTRODUCTION

The Fast Fourier Transform (FFT) is a core block in real-time signal processing, used in systems ranging from FMCW radar [8] and synthetic aperture radar (SAR) [10] to OFDM communications [9]. On FPGAs, fixed-point arithmetic is preferred over floating-point because it needs fewer LUTs, DSP blocks, and BRAMs, while running at a higher clock rate and drawing less power [6].

Choosing the right data word length W is the main design decision in a fixed-point FFT. A shorter word length cuts hardware cost but lets quantization errors accumulate across the $\log_2 N$ butterfly stages, lowering the output SQNR. A longer word length recovers accuracy at the price of more DSP and routing resources. Finding the best trade-off therefore requires an empirical study that models finite-word-length (FWL) effects properly.

A common mistake is to build a reference model that only quantizes the input and then calls a floating-point FFT routine. Such a model tracks the theoretical bound of $6.02W + 1.76$ dB and overestimates real output quality, because it misses the

noise generated inside the FFT by twiddle quantization, post-multiply rounding, and per-stage scaling. This work avoids that pitfall by running the entire FFT in integer arithmetic.

The main contributions of this paper are:

- A bit-accurate Python model that propagates all finite-word-length noise sources through every butterfly stage, covering twiddle quantization, round-half-up after each complex multiply, and fixed $\div 2$ overflow scaling per stage.
- A parameterized Verilog RTL covering $W \in \{8, 10, 12, 14, 16\}$ bits, built on a memory-based single-butterfly architecture.
- A co-simulation flow confirming bit-exact agreement between RTL and the Python model across 200 butterfly vectors and all 1024 output bins for five signal types.
- Post-synthesis resource measurements (LUT, FF, DSP48E1, Block RAM) from Vivado for all five word lengths on the Artix-7 FPGA, revealing a crossover from LUT-based to DSP-based multipliers at $W = 12$ bits.

Section II surveys related work. Section III covers system design. Section IV describes the RTL implementation. Section V presents results. Section VI concludes.

II. RELATED WORK

Research on finite word-length effects in the FFT goes back to the 1970s. Early analytical work [2] derived the $6.02W + 1.76$ dB input-quantization bound and set out rounding noise propagation rules for scaled radix-2 butterflies. Chang and Nguyen [3] extended this to arbitrary radix and rounding modes, noting that twiddle quantization noise must be treated separately from input quantization noise.

Unnikrishnan *et al.* [4] carried out a thorough empirical study measuring SQNR, area, and power as functions of data word length W_D and twiddle word length W_C for $N = 256$ to 1024, confirming the roughly 6 dB/bit improvement rule. That work is the closest published reference to the present study.

On the hardware side, Garrido *et al.* [5], [6] built memory-based single-butterfly FFT cores on Xilinx FPGAs and examined how BRAM and DSP usage scales with word length.

Khairy *et al.* [7] compared streaming and memory-based designs and found that memory-based implementations give the smallest area when throughput requirements are relaxed. The present design follows the same approach.

Fixed-point FFT cores for FMCW radar [8], [9] and SAR [10] all stress the importance of word-length selection to meet system SQNR requirements, which the present paper addresses directly.

III. SYSTEM DESIGN

A. Fixed-Point Number Format

All data values use $Q1.(W-1)$ two's-complement signed format, covering the range $[-1, 1-2^{-(W-1)}]$. Twiddle factors $W_N^k = \cos(2\pi k/N) - j \sin(2\pi k/N)$ use the same format; the value $+1.0$ saturates to the largest positive code.

B. Bit-Accurate Golden Model

Instead of calling a floating-point FFT after quantizing the input, the golden model runs the entire Radix-2 DIT FFT in integer arithmetic, replicating every quantization event that occurs in the hardware:

- 1) **Input quantization:** each sample is rounded to the nearest W -bit code using round-half-up.
- 2) **Twiddle quantization:** cos and sin values are pre-quantized to W -bit codes and stored in ROM initialization files.
- 3) **Complex multiply and round:** the product $t = W_N^k \cdot b$ is computed at full precision and rounded to $W+1$ bits.
- 4) **Butterfly and scale:** outputs $o_0 = (a + t + 1) \gg 1$ and $o_1 = (a - t + 1) \gg 1$ apply a fixed $\div 2$ right shift to prevent overflow.

After $\log_2 N = 10$ stages the result equals $\text{DFT}\{x\}/N$. Output quality is measured by the SQNR defined in (1), where P_{signal} comes from a floating-point FFT reference and P_{noise} is the mean squared error between fixed-point and floating-point outputs.

$$\text{SQNR} = 10 \log_{10} \left(\frac{P_{\text{signal}}}{P_{\text{noise}}} \right) \quad (1)$$

C. Test Signal Types

Six input types cover a range of spectral scenarios: a single-tone sinusoid (bin 50), a two-tone mix (bins 50 and 120), complex Gaussian noise (averaged over 30 independent trials), full-scale DC, a unit impulse, and all-zeros. The all-zeros and impulse cases are excluded from SQNR comparisons because the former has no signal power and the latter underflows after the $\div N$ output scaling.

IV. RTL IMPLEMENTATION

A. Architecture Overview

The design uses a memory-based, in-place, single-butterfly structure. One butterfly datapath handles all $N/2 \times \log_2 N = 5120$ operations in turn, controlled by an FSM. This keeps area low at the cost of throughput, which is acceptable for applications that do not need a continuous output stream. Fig. 1 illustrates the overall software-hardware flow.

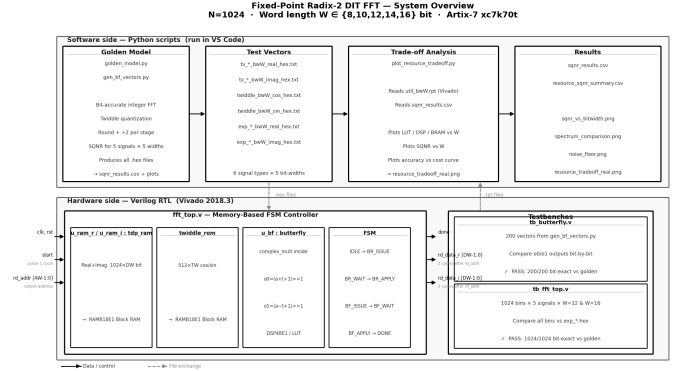


Fig. 1. System overview: the Python golden model (top) generates test vectors and reads Vivado reports; the Verilog RTL (bottom) implements the FFT and is verified bit-by-bit against the golden output.

B. Module Hierarchy

The design consists of four Verilog modules described below and shown in Figs. 2–7.

complex_mult.v (Fig. 2) computes $t = W_N^k \cdot b$ in $Q1.(W-1)$ format. The output holds one extra integer bit ($W+1$ bits total) so that intermediate values do not saturate when $|b|$ is near $\sqrt{2}$. At $W \geq 12$, Vivado maps the multiplications to DSP48E1 blocks; at $W < 12$ it infers LUT-based multipliers, since narrow operands do not fill a 25×18 -bit DSP48E1 block efficiently.

complex_mult.v — Parameterized Fixed-Point Complex Multiplier

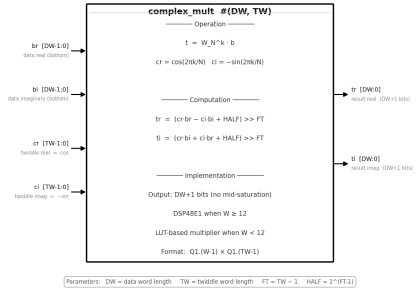


Fig. 2. Block diagram of `complex_mult.v`: parameterized complex multiplier with round-half-up and no intermediate saturation.

butterfly.v (Fig. 3) computes $o_0 = \text{sat}((a + t + 1) \gg 1)$ and $o_1 = \text{sat}((a - t + 1) \gg 1)$. The right-shift by 1 prevents overflow from propagating through the stages. The module is purely combinational and instantiates `complex_mult` internally. Fig. 7(a) shows its Vivado RTL schematic.

twiddle_rom.v (Fig. 4) holds 512 pre-quantized cos and sin entries loaded at startup via `$readmemh`. Setting `rom_style = "block"` tells Vivado to map the ROM to a RAMB18E1 tile.

tdp_ram.v (Fig. 5) is a true dual-port RAM used twice inside `fft_top.v`: as `u_ram_r` for the real-part array and `u_ram_i` for the imaginary-part array. Each port has its own synchronous always block, following the Xilinx template

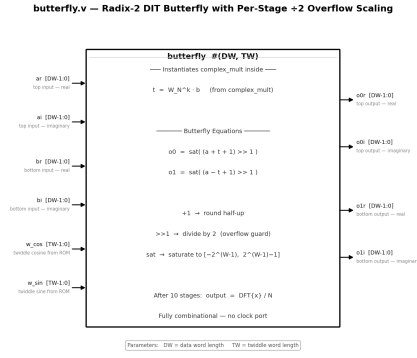


Fig. 3. Block diagram of `butterfly.v`: Radix-2 DIT butterfly with per-stage $\div 2$ overflow scaling, instantiating `complex_mult` internally.

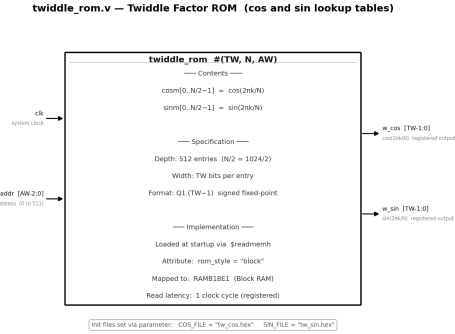


Fig. 4. Block diagram of `twiddle_rom.v`: 512-entry cos/sin ROM in Q1.(W - 1) format, mapped to RAMB18E1.

for RAMB18E1 inference. Fig. 7(b) shows the Vivado RTL schematic of `u_ram_r`.

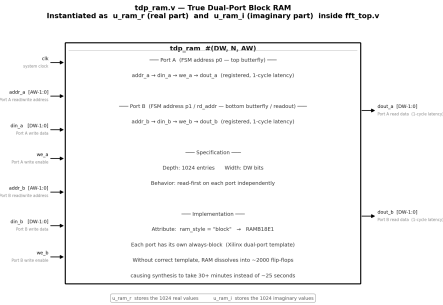


Fig. 5. Block diagram of `tdp_ram.v`: true dual-port Block RAM with independent Port A and Port B, mapped to RAMB18E1.

`fft_top.v` (Fig. 6) connects all sub-modules and drives the FSM through eight states: `S_IDLE`, `S_BR_ISSUE`, `S_BR_WAIT`, `S_BR_APPLY`, `S_BF_ISSUE`, `S_BF_WAIT`, `S_BF_APPLY`, and `S_DONE`. A read-out port (`rd_addr/rd_data_r/rd_data_i`) lets the host scan all 1024 result bins after `done` goes high.

C. BRAM Inference

An early version of `fft_top.v` placed both working arrays in a single always block with multiple write sources.

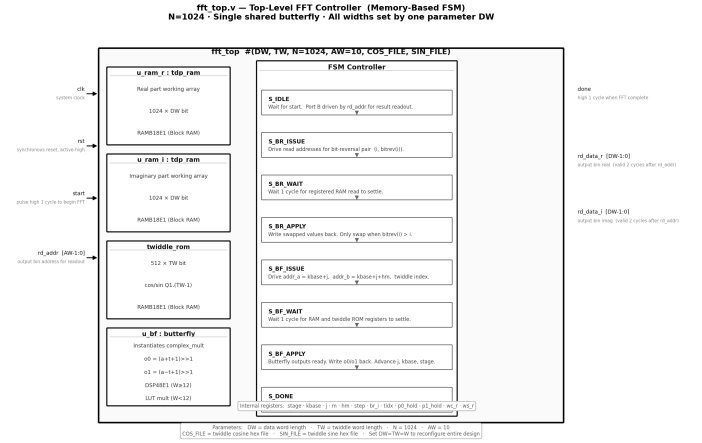


Fig. 6. Block diagram of `fft_top.v`: top-level controller showing sub-module connections and the eight-state FSM.

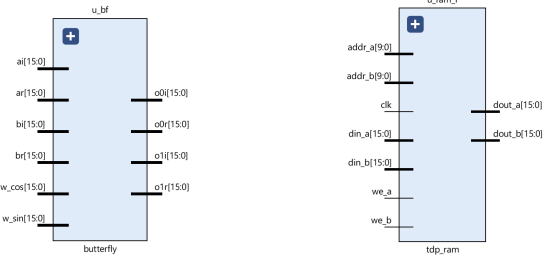


Fig. 7. Vivado RTL schematics of two synthesized instances showing all port connections.

Vivado could not infer this as a RAM and dissolved it into roughly 2000 flip-flops, making synthesis take over 30 minutes. Moving the RAM into a dedicated `tdp_ram` module with one always block per port and adding `ram_style = "block"` fixed the issue; synthesis time dropped to about 25 seconds.

D. Co-Simulation Verification

Verification runs in two passes. First, 200 random butterfly vectors from the golden model are applied to `tb_butterfly.v`; all 200 output pairs match the reference exactly. Second, `tb_fft_top.v` loads the working RAM via hierarchical references, runs the full 1024-point FFT, and checks every output bin against the golden expected values. This test passes for five signal types at both $W = 12$ and $W = 16$ bits, confirming bit-exact agreement with the Python model. All resource numbers in Section V are from post-synthesis analysis in Vivado; physical board testing is left as future work.

V. RESULTS AND DISCUSSION

A. SQNR Results

Table I lists the output SQNR from the golden model for four signal types across all five word lengths. The theoretical input-only bound $6.02W + 1.76$ dB is included for reference.

TABLE I
OUTPUT SQNR (dB) — $N = 1024$, SCALED RADIX-2 FFT

Signal	W=8	W=10	W=12	W=14	W=16
Single tone	10.2	22.1	34.4	46.3	58.4
Multi tone	7.2	19.8	31.2	43.5	55.4
Full-scale DC	15.3	27.3	39.4	51.4	63.4
Random noise	2.5	14.5	26.5	38.5	50.6
Input-only bound	49.9	62.0	74.0	86.0	98.1

SQNR rises by roughly 6 dB per added bit, consistent with theory. All measured values sit about 40 dB below the input-only bound. That gap is the processing noise from twiddle quantization and rounding across the ten butterfly stages — exactly what a naive input-only model misses.

B. Hardware Resource Results

Table II shows the post-synthesis utilization from Vivado on the Artix-7 xc7k70tbfv676-1.

TABLE II
POST-SYNTHESIS RESOURCE UTILIZATION — ARTIX-7
(XC7K70TFBV676-1)

W (bit)	LUT	FF	BRAM	DSP48E1
8	581	138	2	0
10	829	146	2	0
12	341	154	2	5
14	366	162	2	5
16	383	170	2	5

A crossover occurs between $W = 10$ and $W = 12$. Below 12 bits, operands are narrow enough that Vivado builds the multiplier from LUTs with DSP = 0. From 12 bits onward, five DSP48E1 blocks are inferred and the LUT count drops sharply from 829 to 341. The BRAM count stays at 2 for all widths because both data arrays and the twiddle ROM fit within two RAMB18E1 tiles.

C. Trade-off Analysis

Fig. 8 plots the SQNR against word length for all four signal types. All measured curves fall about 40 dB below the input-quantization-only bound (dashed), confirming that in-FFT processing noise dominates.

Fig. 9 shows the LUT, FF, DSP, and BRAM usage across all five configurations. The crossover at $W = 12$ is annotated on the right subplot.

Fig. 10 plots combined hardware cost (LUT + 100 × DSP48E1) against single-tone SQNR. The 12-bit point sits at the knee of the curve with 34.4 dB SQNR at 841 cost units. For FMCW radar and similar applications requiring 30–40 dB SQNR [8], [9], this configuration provides sufficient accuracy at the lowest hardware cost among all DSP-based designs tested.

Fig. 11 shows the output spectrum at each word length. At $W = 8$ the noise floor sits near -45 dBFS, which would mask closely spaced spectral components. At $W = 16$ it falls below

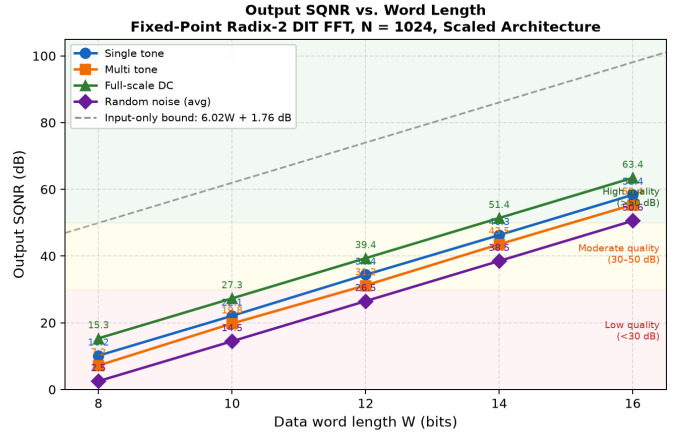


Fig. 8. Output SQNR vs. word length ($N = 1024$). Shaded bands indicate quality levels. All curves fall about 40 dB below the theoretical input-quantization-only bound (dashed line).

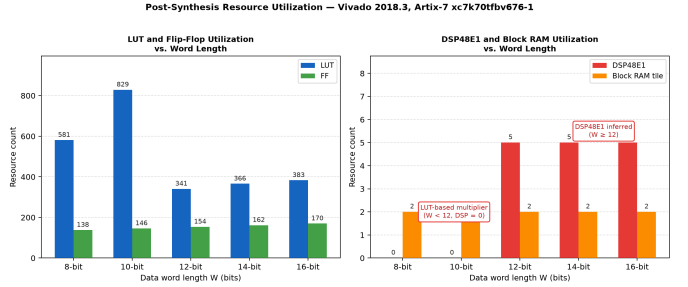


Fig. 9. Post-synthesis utilization on the Artix-7 xc7k70tbfv676-1. Left: LUT and FF counts. Right: DSP48E1 and BRAM tile counts, with the crossover at $W = 12$ annotated.

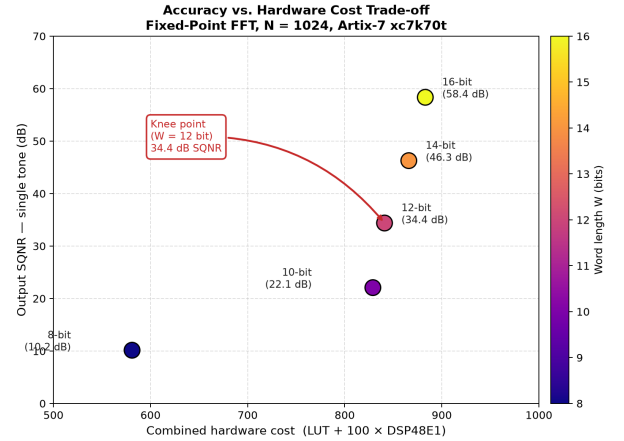


Fig. 10. Accuracy vs. hardware cost. The 12-bit point (annotated) is at the knee of the curve: 34.4 dB SQNR at the lowest cost among DSP-based configurations.

-90 dBFS. Applications requiring good dynamic range, such as radar Doppler processing, should use $W \geq 12$ to keep the floor below -65 dBFS.

Fig. 12 shows the per-bin quantization error relative to the $W = 16$ reference. The error is roughly flat across the

Single-Tone Output Spectrum vs. Word Length
N = 1024-point FFT | Input: single tone at bin 50

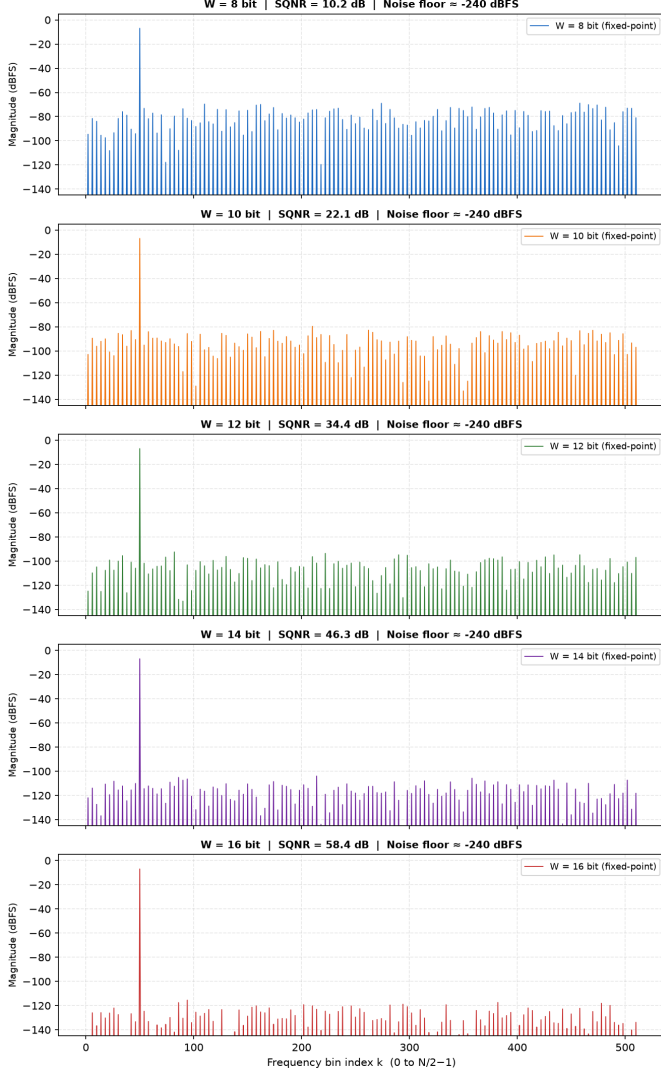


Fig. 11. Single-tone output spectrum (tone at bin 50, $N = 1024$) for $W = 8$ to 16 bits. The noise floor drops by roughly 12 dB for every 2-bit increase in word length.

full frequency range at every word length, confirming that the quantization noise in this scaled FFT is spectrally white. A single SQNR number therefore gives a consistent noise-floor guarantee across all output bins, not just at specific frequencies.

Per-Bin Quantization Noise Floor vs. Word Length
Error = fixed-point output - $W=16$ reference | Single tone at bin 50, $N = 1024$

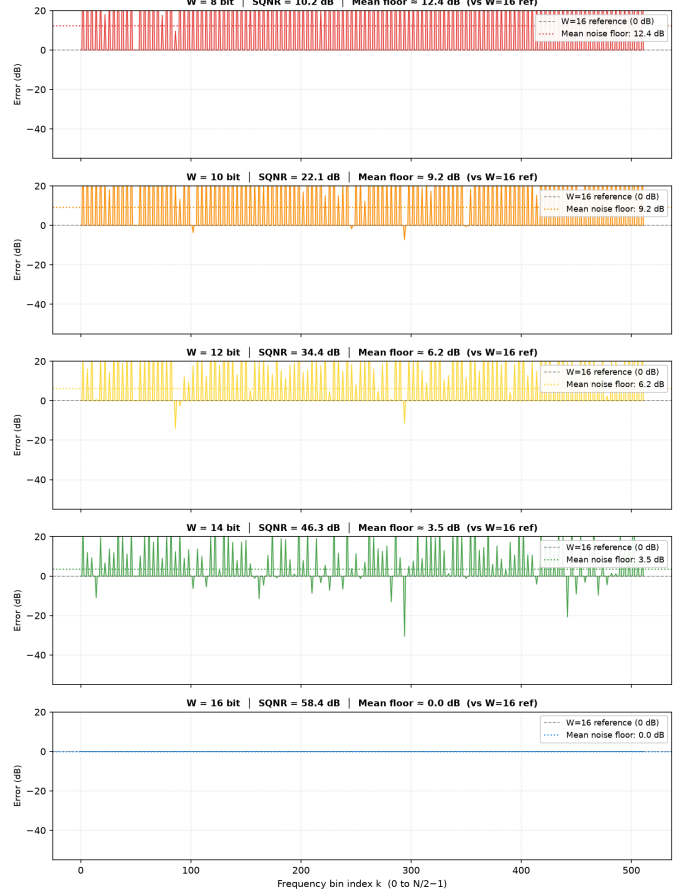


Fig. 12. Per-bin quantization error relative to the $W = 16$ reference. The spectrally flat noise supports using SQNR as a scalar quality metric for this design.

VI. CONCLUSION

This paper studied bit-width trade-offs in a fixed-point Radix-2 DIT FFT on the Xilinx Artix-7 FPGA, covering word lengths from 8 to 16 bits. The key contribution is a bit-accurate Python model that propagates quantization noise through every butterfly stage rather than only at the input, giving a realistic SQNR estimate. The model shows that the true output SQNR sits roughly 40 dB below the commonly cited input-quantization bound, a gap that would be missed by a simpler input-only approach.

Co-simulation confirmed that the parameterized Verilog RTL is bit-exact with the Python model across all 1024 output bins for five signal types. Synthesis on the Artix-7 reveals a crossover at $W = 12$ bits where Vivado switches from LUT-based to DSP-based multipliers, dropping the LUT count from 829 to 341. The 12-bit configuration achieves 34.4 dB SQNR with five DSP48E1 blocks, two BRAM tiles, and 341 LUTs, making it the recommended choice for moderate-quality applications such as FMCW radar and OFDM receivers.

Future work will explore block-floating-point scaling as a

replacement for the fixed per-stage scaling used here, which could recover several decibels of SQNR at short word lengths. The study will also extend to larger FFT sizes ($N = 4096$, $N = 16384$) and pipelined architectures.

ACKNOWLEDGMENT

The authors thank the Faculty of Semiconductor and Automotive Digital Design at FPT University for providing laboratory access and computational resources.

REFERENCES

- [1] J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex Fourier series," *Math. Comput.*, vol. 19, no. 90, pp. 297–301, Apr. 1965.
- [2] A. V. Oppenheim and R. W. Schaffer, *Discrete-Time Signal Processing*, 3rd ed. Upper Saddle River, NJ: Pearson Prentice Hall, 2010.
- [3] W. Chang and T. Q. Nguyen, "On the fixed-point accuracy analysis of FFT algorithms," *IEEE Trans. Signal Process.*, vol. 56, no. 10, pp. 4673–4682, Oct. 2008.
- [4] N. K. Unnikrishnan, M. Garrido, and K. K. Parhi, "Effect of finite word-length on SQNR, area and power for real-valued serial FFT," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, 2019.
- [5] M. Garrido, "Challenging the limits of FFT performance on FPGAs," Linköping University, Tech. Rep., 2016. [Online]. Available: <https://liu.diva-portal.org/smash/get/diva2:927920/FULLTEXT01.pdf>
- [6] M. Garrido, K. K. Parhi, and P. Löfgren, "A 1 million-point FFT on a single FPGA," Linköping University, Tech. Rep., 2019. [Online]. Available: <https://liu.diva-portal.org/smash/get/diva2:1367440/FULLTEXT01.pdf>
- [7] A. Khairy *et al.*, "Comparison of streaming and memory-based FFTs for FPGA implementation," Linköping University, Tech. Rep., 2024. [Online]. Available: <https://www.diva-portal.org/smash/get/diva2:1885023/FULLTEXT01.pdf>
- [8] I. Brayda *et al.*, "Design space exploration of FFT architectures and numerical formats for SoC-based FMCW radar signal processing," *IEEE Access*, vol. 13, 2025.
- [9] J. Tian *et al.*, "FPGA implementation of an efficient FFT processor for FMCW radar applications," *Sensors*, vol. 21, no. 19, p. 6444, 2021.
- [10] L. Geng *et al.*, "An FPGA-based four-channel 128k-point FFT processor suitable for spaceborne SAR," *Electronics*, vol. 10, no. 7, p. 816, 2021.
- [11] Y. Ji *et al.*, "Fixed-point FPGA implementation of the FFT accumulation method for real-time cyclostationary analysis," *ACM Trans. Reconfigurable Technol. Syst.*, vol. 15, no. 3, 2022.
- [12] P. Löfgren and M. Garrido, "SFF: the single-stream FPGA-optimized feedforward FFT hardware architecture," *J. Signal Process. Syst.*, 2015. [Online]. Available: <https://d-nb.info/1164027603/34>
- [13] S. Uzun *et al.*, "Finite word length effect in practical block-floating-point FFT," in *Proc. SIGNAL 2025*, 2025. [Online]. Available: https://www.thinkmind.org/articles/signal_2025_2_60_60044.pdf
- [14] Xilinx Inc., *7 Series FPGAs Data Sheet: Overview*, DS180, Rev. 2.6. San Jose, CA: Xilinx, 2020.